PALESTINIAN ROOTS PLATFORM

جذور

Juthoor
System Design Report

Functional and technical requirements, system architecture, technology stack, frontend design, backend and database, and process flows.

Version	1.0
Date	21 October 2026
Source	github.com/yossefbouz/juthoor, main branch
Flow reference	Process Flow Diagram v1.2, reviewed 11 October 2026
Audience	Project team, partners and reviewers

Every figure and status in this report was read from the repository code, database migrations and project logs on the date above. Items marked “Partly built” or “Not built” are open work, listed with suggested actions in section 10.

CONTENTS

What this report covers

1 Summary
2 Functional requirements
3 Technical requirements
4 System architecture
5 Technology stack
6 Frontend design
7 Backend and database
8 Process flows
9 Quality, testing and delivery
10 Gaps, risks and next steps
A Database table inventory
B Glossary

How to read the status marks

Built	Present in the code on the main branch and reachable in the app.
Partly built	Exists, but does not yet meet the full requirement. The gap is named in the same row.
Not built	Required by the process flow or specification, with no implementation in the code.

1 · SUMMARY

Juthoor at a glance

Juthoor, Arabic for “roots”, is a bilingual genealogy platform that helps Palestinians worldwide rebuild, connect and preserve their family trees. It is Arabic first and right to left, follows the GEDCOM 7 genealogy standard, and links separate family trees into one shared Palestine Tree through a probabilistic matching engine.

33 pages in the web app

36 database tables

48 database migrations

110 row-level security policy definitions

57 SQL functions, including the matching engine

456 Palestinian villages and places seeded

116 unit test cases, plus 6 end-to-end specs

17 database test files

State of the build

- **Core family trees are built.** Users can create trees by hand or import GEDCOM files, edit people and relationships on a visual chart, and export the tree again.
- **Search is built.** Family Finder searches across trees with Arabic normalisation, phonetic matching and Latin to Arabic transliteration.
- **The matching engine is built and set to review only.** Automatic merging is off by default. Matches go to an admin queue or to both tree owners.
- **Privacy for living people is partly built.** Search, the people list and the relationship path hide living people’s details. The GEDCOM export does not yet.
- **Contact, moderation and access logging are not built yet.** They are listed in section 10 together with the decisions they depend on.

What the platform must do

Requirements follow the eight modules of Process Flow Diagram v1.2, plus the specification rules for tree building and matching. Status is taken from the code.

ID	Requirement	Where in the app	Status
1.0 Sign in and register			
1.1	Register with email, confirmed by an 8-digit email code	/sign-up	Built
1.2	Sign in by password, email code or social account; reset a forgotten password	/login, /forgot-password	Built
1.3	Verify identity with a government ID and family evidence, reviewed by an administrator	/verify, /admin/verifications	Built
1.4	On registration, find the new user in the Palestine Tree and link the account after confirmation	Add-yourself page exists; no automatic match	Not built
2.0 Home and information			
2.1	Home page, Who we are, Why we are doing this, How it works, Contact us	/, /about, /why, /how, /contact	Built
2.2	Privacy policy and terms of use, in Arabic and English	/privacy, /terms	Built
3.0 Trees			
3.1	List the trees a user owns or belongs to	/tree, /dashboard	Built
3.2	View and edit a tree on a chart and in a 360° person view	/tree/[id]/chart, /person/[id]	Built
3.3	Horizontal, vertical and fan chart layouts	One chart layout today	Partly built
3.4	Create a tree by hand or by importing a GEDCOM file up to 5 MB	/dashboard/new, import dialog	Built
3.5	Export a tree as a GEDCOM file, leaving out living people	Export works; no living filter	Partly built
3.6	Access rights: request read or write access, with proof of family for write; owner approves	/dashboard/requests	Built
3.7	Palestine Tree: one combined view over all linked trees	Link overlay built; no combined chart	Partly built
4.0 Individual search			
4.1	Search by first, father’s, grandfather’s and family name, with phonetic matching	/search	Built
4.2	Filter by village or town and by district	/search place filter	Built
4.3	See how you are connected to a person	Degrees path on search	Built
4.4	View achievements and contribution to the cause	Person story panel	Built
4.5	View a person’s photos and documents	Person attachments	Built
4.6	Reach a person’s family through a request, not by phone number	Relay table only	Not built
5.0 Family search and 6.0 Village search			
5.1	Search by family name and open a family page	/families, /families/[surname]	Built
5.2	Family photo gallery and document archive	No screen	Not built

ID	Requirement	Where in the app	Status
6.1	List villages and towns with a district filter; open a village page with history and photos	/villages, /villages/[id]	Built
7.0 Pictures and 8.0 Documents			
7.1	Upload photos and documents to a person, tagged by type, in private storage	Person page	Built
7.2	Search all pictures and all documents by name, place and date	No screen	Not built
Specification rules for tree building and matching			
R1	Record women under their maiden name	Add-person form	Built
R2	Every child is linked to a mother; an unknown mother gets a placeholder that can be filled in later	Add-relative action	Built
R3	When a man has several wives, pick the right mother for each child	Mother picker	Built
R4	A child inherits the father’s surname and village by default	No default today	Not built
R5	Matching runs as an end-of-day batch over 25 weighted fields	Batch built; no schedule set	Partly built
R6	Scores of 450 or more merge automatically; lower scores go to admin review	Switch built, off by design	Partly built
R7	Admin resolves each match: merge, reject or defer; owners are notified	/admin/review, in-app notices	Built

3 · TECHNICAL REQUIREMENTS

How the platform must behave

Non-functional requirements for security, privacy, language, standards, performance and operations.

Area	Requirement	How it is met today	Status
Security	Every table is protected at the database layer	Row-level security on all application tables, with shared access helpers such as can_access_tree and is_admin	Built
Security	Users cannot raise their own permissions	A trigger blocks changes to the admin flag through the public API. The migration is committed with this report; confirm it is applied in production	Built
Security	Identity documents are kept apart and private	Private storage bucket; only the uploader and administrators can read	Built
Privacy	Living people are private by default	Masked in search, the people list, relationship paths and match cards. Not yet in the GEDCOM export or the tree chart loader	Partly built
Privacy	Every view of private data is logged	Only match decisions are logged	Not built
Language	Arabic first, right to left, with a full English version	Arabic layout by default, locale switch, bilingual copy and legal texts	Built
Standards	GEDCOM 7 import and export without losing record identifiers	Parser and serializer keep original record IDs and create new ones without collisions	Built
Search	Arabic name variants and Latin spellings find the same person	Arabic normalisation, phonetic keys, trigram similarity and a Latin to Arabic transliteration table	Built
Performance	95% of pages load in under 2 seconds	Cached auth claims and bulk tree loading; not yet measured in production	Partly built
Availability	99.9% uptime	Single server with a health check on deploy; no redundancy or uptime monitoring	Not built
Mobile	Usable from 360 px wide screens, for users in camps	Responsive layout; not formally tested at 360 px	Partly built
Accessibility	WCAG 2.1 AA	Accessible Radix UI components; no audit yet	Partly built
Compliance	Data protection review before public launch	Privacy policy drafted; the privacy contact email is still a placeholder; hosting region not documented	Partly built
Delivery	Every change is checked and deployed automatically	CI runs type-check, lint and build; deploy runs after CI passes on main. Database migrations are applied by hand	Built

4 · SYSTEM ARCHITECTURE

Three tiers and a delivery pipeline

The browser talks to a Next.js server on a private server. The Next.js server talks to Supabase, which holds the database, sign-in, file storage and live updates. Most business rules live inside the database, so they apply no matter which client calls it.

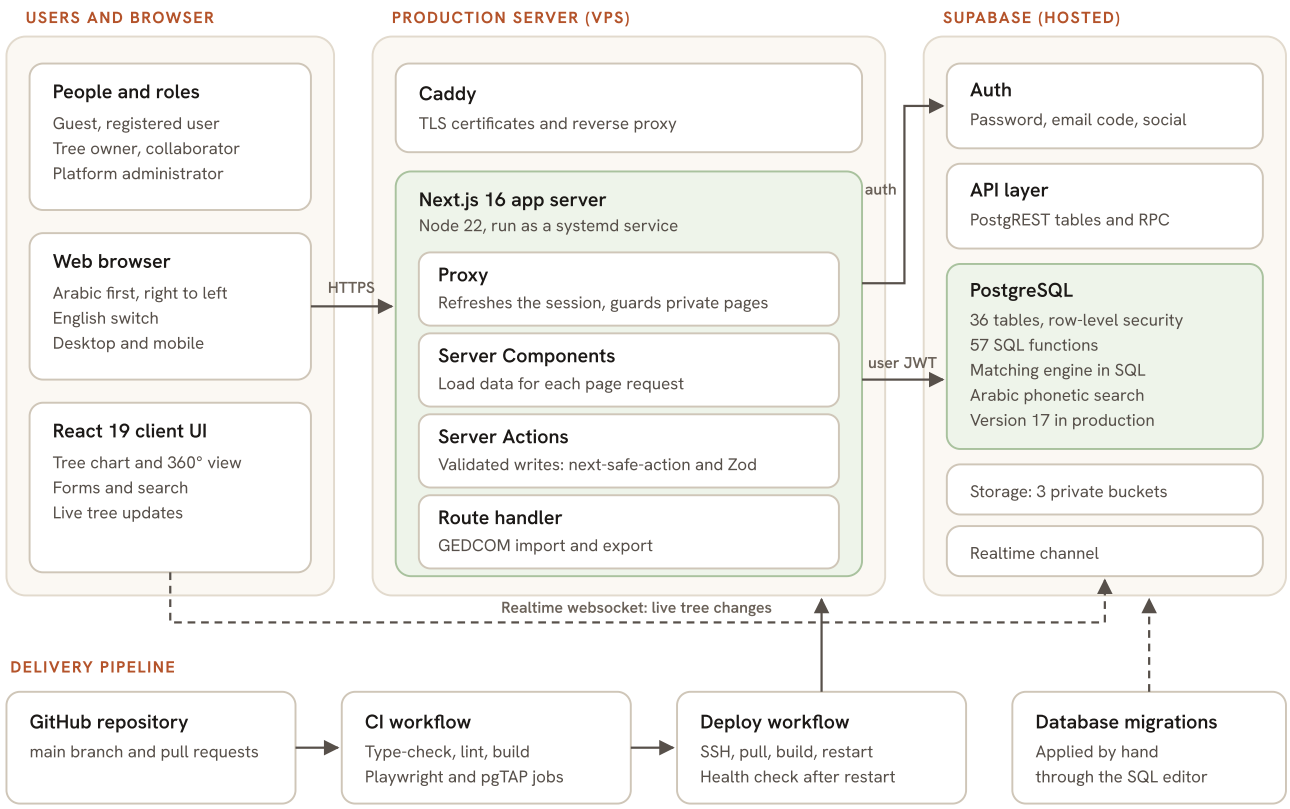


Figure 1. Runtime components and the delivery pipeline. Solid arrows are requests; dashed arrows are live or manual connections.

How a request flows

1. **The browser calls the server.** Caddy terminates TLS and passes the request to Next.js.
2. **The proxy checks the session.** It refreshes the Supabase session cookie and sends signed-out visitors away from private pages.
3. **Server code calls Supabase as the user.** Data loaders and server actions use the user's token, so every query passes through row-level security.
4. **The database enforces the rules.** Policies decide which rows the user may see. Security-definer functions handle sensitive steps such as search masking, access approvals and match decisions.
5. **The page renders on the server.** Interactive parts such as the tree chart hydrate in the browser and subscribe to live changes.

Trust boundaries

- **The web app holds no service key.** No code in the web app uses the Supabase service role, so a server bug cannot bypass row-level security.
- **Administrators are a database flag.** Admin pages check the flag on the server, and the database refuses any attempt to set it through the API.
- **Files are private.** Attachments, proof of family and identity documents sit in three private buckets with their own access policies.

5 · TECHNOLOGY STACK

What we are using

A TypeScript monorepo managed by pnpm and Turborepo. The web app is in `apps/web`; the database schema, tests and seeds are in `apps/database`.

Frontend

Layer	Technology	Version	Role
Framework	Next.js, App Router	16.2	Server rendering, routing, server actions, route handlers
UI library	React	19.2	Components and client interactivity
Language	TypeScript	6.0	Typed code across app and tests
Styling	Tailwind CSS	4.1	Utility styling with right-to-left support and design tokens
Components	shadcn/ui on Radix UI, lucide icons	latest	Accessible dialogs, menus, tabs, sheets, sidebar
Forms	react-hook-form with Zod	7.72, 4.3	Form state and validation shared with the server
Server calls	next-safe-action	7.10	Typed, validated server actions with friendly bilingual errors
Data caching	TanStack Query	5.96	Client-side caching where pages need it
Tree layout	relatives-tree and a custom chart	3.2	Family chart with pan, zoom and connectors
Genealogy	parse-gedcom and an in-house serializer	2.0	GEDCOM 7 import and export
Arabic text	jslingua and in-house phonetic rules	0.13	Transliteration and name folding in the browser
Motion and charts	Framer Motion, Recharts	12, 2.15	Animation and statistics
Fonts	Noto Kufi Arabic, Hanken Grotesk, Amiri, Cairo	Google Fonts	Arabic and Latin typography

Backend and data

Layer	Technology	Version	Role
Platform	Supabase, hosted	managed	Database, auth, storage, realtime, API
Database	PostgreSQL	17 prod, 15 local	Schema, row-level security, business logic in SQL
Extensions	pg_trgm, unaccent, uuid-ossp	bundled	Similarity search, accent folding, identifiers
Auth	Supabase Auth with @supabase/ssr	0.10	Password, email code and social sign-in with cookie sessions
Client SDK	supabase-js	2.101	Queries, RPC calls, storage and realtime
Runtime	Node.js	22+	Runs the Next.js server

Tooling and delivery

Layer	Technology	Version	Role
Monorepo	pnpm workspaces, Turborepo	9, 2.2	Package management and task running
Unit tests	Vitest with Testing Library	4.1	Logic and component tests
End-to-end tests	Playwright	1.51	Browser tests against a local Supabase stack
Database tests	pgTAP	bundled	Tests for policies, masking and the matching engine
Lint	oxlint	1.58	Fast lint in CI
CI and deploy	GitHub Actions	hosted	Checks on every push; deploy to the server after checks pass on main
Hosting	Linux VPS with Caddy and systemd	self-managed	Runs the web app behind TLS

6 · FRONTEND DESIGN

How the web app is built

The frontend is organised in four layers. Pages render on the server by default; only interactive parts run in the browser.

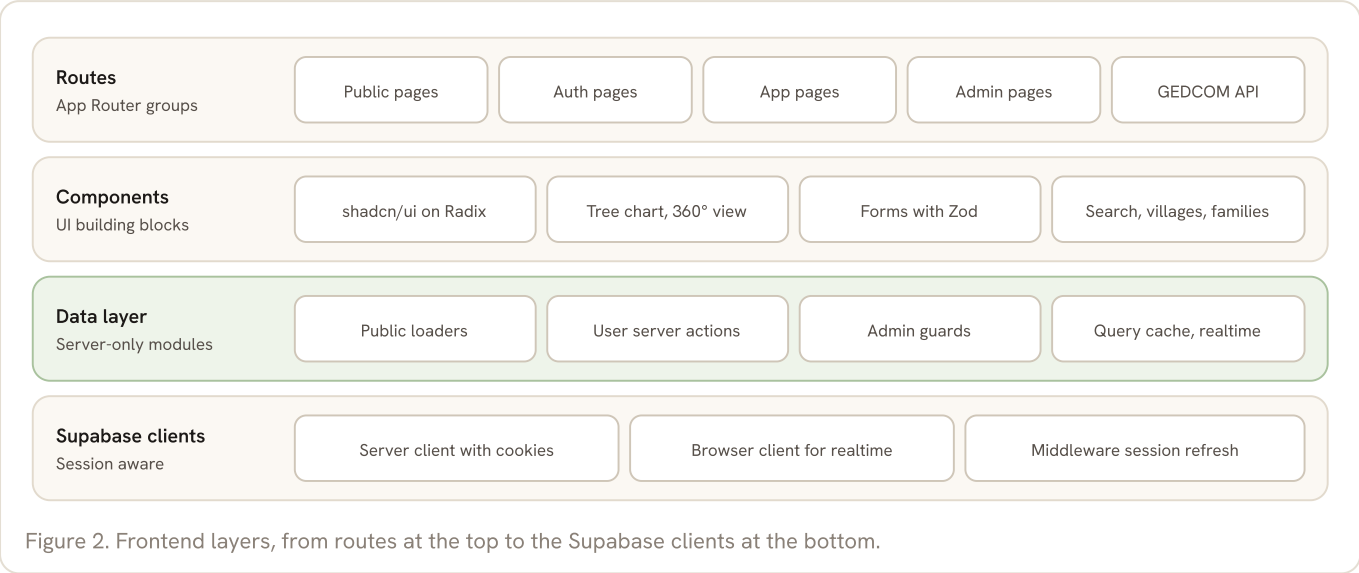


Figure 2. Frontend layers, from routes at the top to the Supabase clients at the bottom.

Route map

Area	Routes	Who can open them
Public	/, /about, /why, /how, /contact, /privacy, /terms	Everyone
Sign-in	/login, /sign-up, /forgot-password, /update-password, /auth/auth-code-error	Everyone
Dashboard	/dashboard, /dashboard/new, /dashboard/requests, /dashboard/connections	Signed-in users
Trees	/tree, /tree/[id], chart, people, person/[id], add-person, add-self	Signed-in users with tree access
Discovery	/search, /families, /families/[surname], /villages, /villages/[id]	Signed-in users
Verification	/verify	Signed-in users
Admin	/admin/review, /admin/verifications	Administrators only
API	/api/tree/[id]/gedcom: GET exports, POST imports	Signed-in users, checked by row-level security

Design system

Colours are defined once as tokens and switch for dark mode. The palette is olive, gold, terra and stone on a warm cream background.

Olive	Gold	Terra	Cream
#3E7030	#D4A31E	#B4532A	#F7F3ED

Arabic text uses Noto Kufi Arabic with Cairo and Amiri as fallbacks. Latin text uses Hanken Grotesk with Inter as fallback.

Language and direction

The page starts in Arabic with right-to-left layout. A locale switch changes copy and direction. Layout uses logical start and end classes so the sidebar and panels mirror correctly in both languages.

Key screens

- **Tree workspace.** Chart with pan and zoom, add-relative buttons on each card, ghost cards for missing parents.
- **360° person view.** Parents above, children below, siblings and spouses beside.
- **Admin review.** Side-by-side match cards with agree, disagree and missing marks.

Where the data lives and how it is protected

The database follows the GEDCOM 7 model: people, names, events, families and places. Around that core sit access control, media, verification and the matching engine.

Tables by domain

Domain	Tables	Purpose
Genealogy core	trees, persons, person_names, events, families, family_children, places	The family record in GEDCOM shape
Users and access	profiles, tree_members, identity_verifications	Accounts, roles, access requests, ID checks
Content and media	person_profiles, person_attachments, place_profiles	Stories, photos, documents, village pages
Search	match_paths, name_variants	Cached relationship paths, name dictionary
Matching	match_features, match_features_dirty, match_block_keys, match_block_skips, matches, matching_runs, app_settings, person_privacy_holds, eval_pairs, eval_runs	Feature vectors, candidate pairs, scores, runs, settings, opt-outs, evaluation
Linking and review	person_links, merge_log, match_hints, match_audit, notifications, contact_relay, overlay_refresh_queue	Confirmed links, undo log, owner hints, audit, notices, anonymous relay

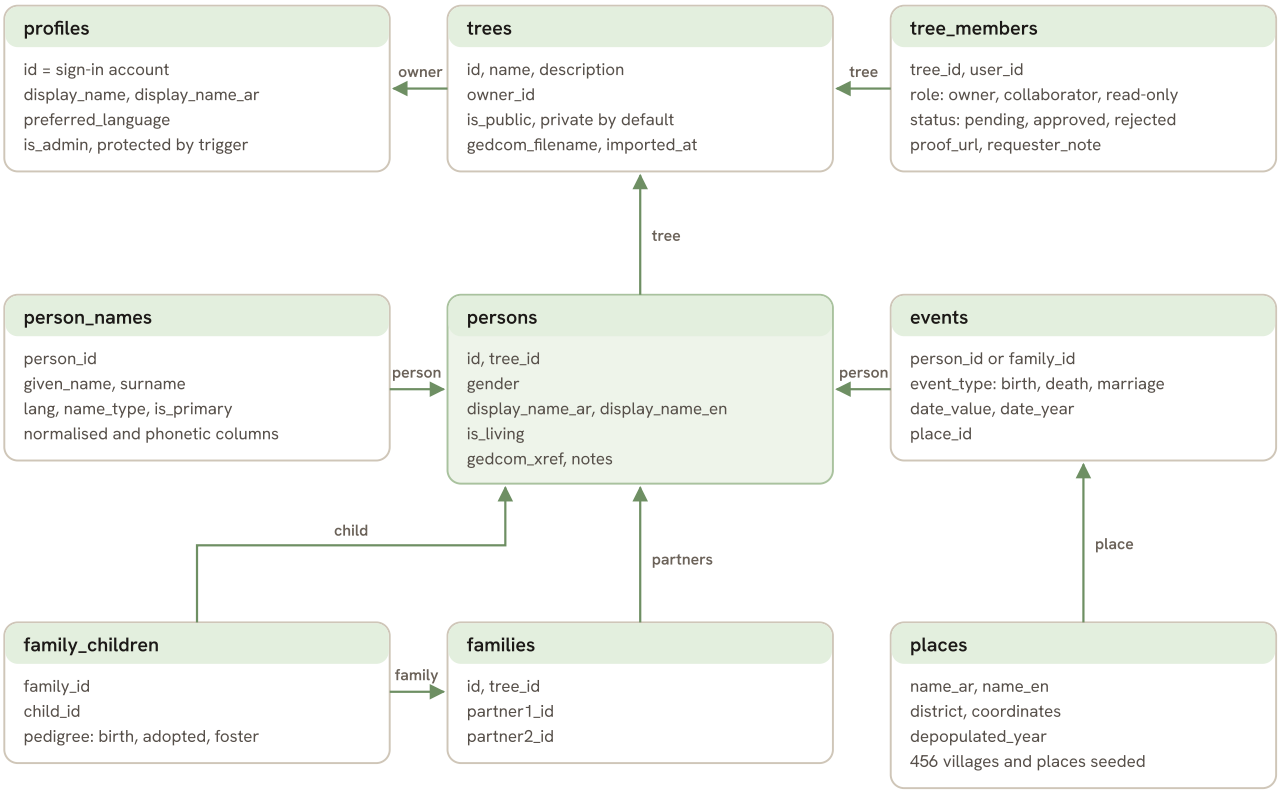


Figure 3. Core genealogy tables. Arrows point from a table to the table it refers to.

Security, search and storage

Security model

Mechanism	What it does
Row-level security	Every application table has policies. Tree data is readable when the tree is public, owned by the user, or the user is an approved member. Writes need owner or collaborator rights.
Access helpers	can_access_tree, can_write_tree, is_tree_owner, is_tree_member and is_admin are shared by the policies.
Definer functions	Search, access approvals, match decisions and link changes run as security-definer functions with a pinned search path. Each one checks the caller itself.
Living-person masking	is_person_living treats anyone without a death record and born less than a century ago, or with no birth year, as living. Search hides their birth year and village from non-members; relationship paths show “Hidden relative”.
Match privacy	Owners never read the raw match table. They see a masked card view. Pairs with a living person are never merged by an admin; both owners must accept.
Admin guard	A trigger blocks any change to the admin flag made through the public API.
Storage policies	Files live under a folder named after the uploader. Policies allow the uploader, the tree owner or administrators, depending on the bucket.

Family Finder search

Search runs in one database function. It normalises Arabic spelling, builds phonetic keys and compares names with trigram similarity. It scores given name, family name, father, mother, birthplace and birth year, and returns a score breakdown. Private trees are redacted, and living people lose their birth year and village unless the searcher is the owner or an approved member.

Storage buckets

Bucket	Contents	Who can read
person-attachments	Photos and documents tagged to a person, such as portraits, certificates and land deeds	Anyone who can read that person’s tree
proof-of-family	Evidence uploaded with a write-access request	The requester and the tree owner
verification-docs	Government IDs and family evidence for identity checks	The uploader and administrators

The matching engine

The matching engine finds the same person recorded in different family trees. It is written in SQL and follows the Fellegi-Sunter method used in record linkage.

Stage	What happens
1. Change queue	Triggers on people, names, events and families put the person and close relatives into a queue whenever data changes.
2. Features	Each queued person gets a feature vector: transliterated, normalised and phonetic names for self, parents, grandparents and spouses, plus life facts.
3. Blocking	Several blocking passes pick candidate pairs from different trees. Caps stop oversized groups from slowing the run.
4. Scoring	score_pair weighs 25 fields. Hard vetoes reject pairs with a different gender, the same tree, birth years more than 15 years apart, or a privacy hold.
5. Routing	Pairs with a living person become masked hints for both owners. Other pairs go to the admin review queue. Automatic merging is switched off by default.
6. Linking	A confirmed link joins two records. Links form the Palestine Tree overlay. Any link can be revoked, which removes its notifications, relay tokens and cached paths.

> **Open point.** The batch function exists, but no schedule is defined in the repository. Running it nightly needs a scheduled job in Supabase or on the server.

Environments and migrations

- **Local.** The Supabase CLI runs the full stack in Docker. A reset builds all 48 migrations from scratch, and a demo seed creates trees, people, matches and verifications.
- **Production.** Hosted Supabase on PostgreSQL 17. Migrations are applied by hand after testing locally. The security fixes of 12 October 2026 are now recorded in the migrations folder.
- **Types.** The TypeScript database types are maintained by hand. Running the type generator overwrites them, so each new table is added manually.

The eight modules and their status

This map follows Process Flow Diagram v1.2. Each line is one step from the diagram, marked by what the code does today.



Figure 4. Module map from Process Flow Diagram v1.2 with the build status of each step.

How the flow review comments are reflected

Comment	Point raised on 11 October 2026	Where the build stands
G1	Define PFT, IFT and VCC	Defined in the glossary, appendix B
G2	Use one name per role and list all roles	Roles in the code: guest, registered user, tree owner, collaborator, read-only member, administrator. No moderator role
#4	Anyone could claim to be an existing person	No claim flow exists. Identity checks are per account; write access needs owner approval
#6	Social login, sign out and delete account	Social sign-in and sign out are built. Account deletion is not
#8	Privacy policy, terms, FAQ and a language switch	Privacy policy, terms and the switch are built. No FAQ page
#18	Choose the district first, then the village	The village picker shows each district but does not filter by it first
#19	Search results must respect privacy for living people	Built: birth year and village are hidden from non-members
#20	One access check with an access-denied branch	Built as one check with a request-access path, figure 6
#21	Show the shortest path instead of every connection	Built: the shortest path, with hidden names for living people

Three key flows

A. Registration and identity verification

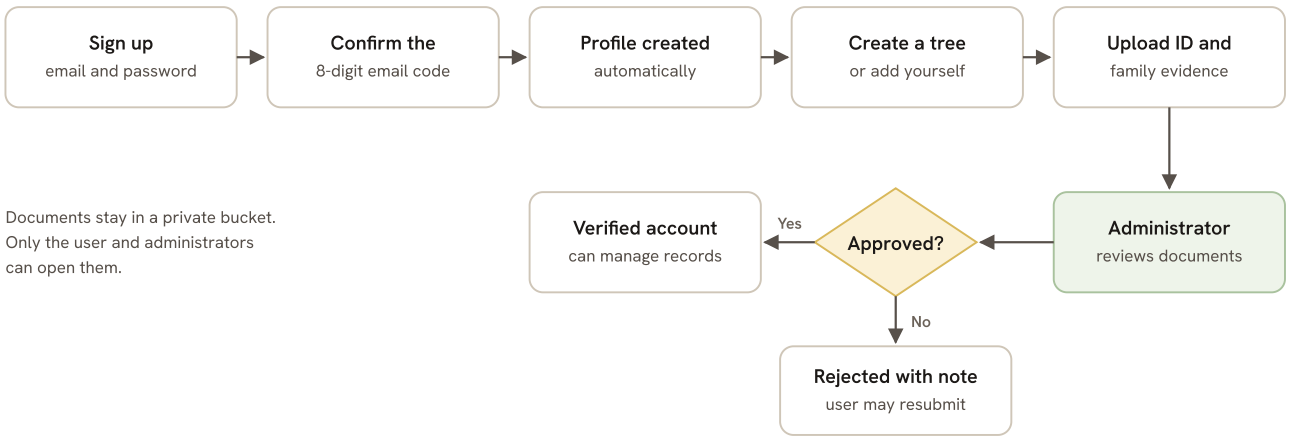


Figure 5. Process steps 1.1 and 1.2 as built. Matching a new user to an existing person, step 1.1.2, is not built.

B. Requesting access to a family tree

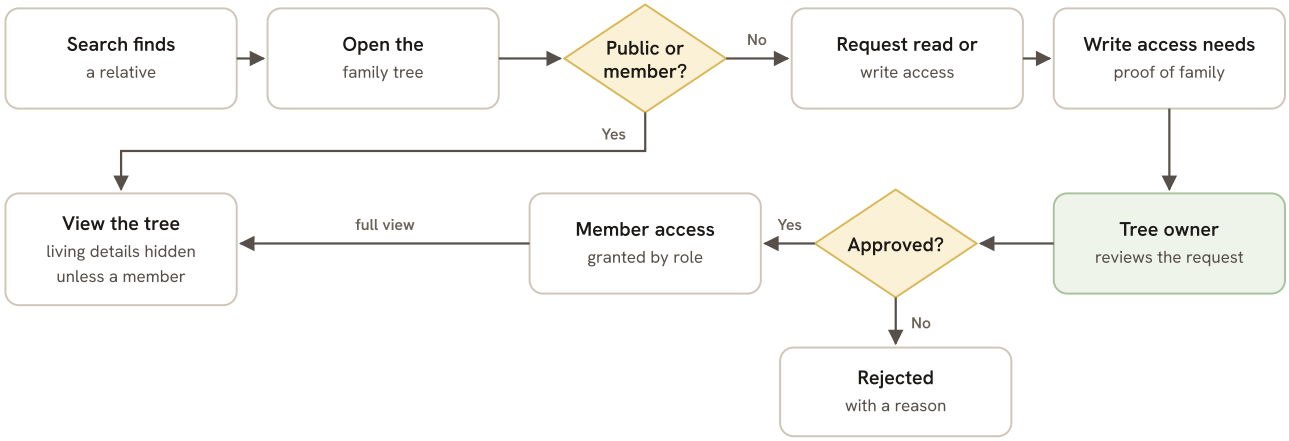
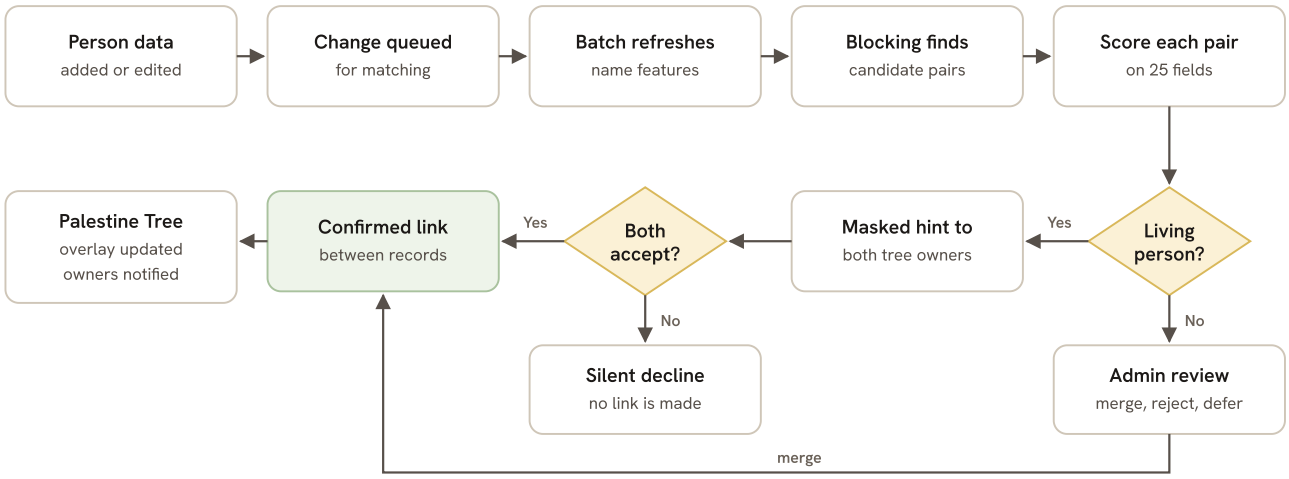


Figure 6. Access requests. The access check sits right after a tree is opened, as the flow review asked.

Matching flow and roles

C. Matching records across trees



Automatic merging is switched off by default. High scores still go to review until precision is measured.

Figure 7. The matching flow. Any confirmed link can be revoked later, which removes it from the overlay.

Roles and what each can do

Role	Who	Can do
Guest	Not signed in	Read the public information pages and sign up
Registered user	Any signed-in account	Create trees, search, verify identity, request access to other trees
Tree owner	The creator of a tree	Everything on the tree; approve or reject access requests; accept or decline match hints; place privacy holds; import and export
Collaborator	Approved member with write access	Add and edit people, relationships, stories, photos and documents
Read-only member	Approved member with read access	See the full tree, including details of living people
Administrator	Account with the admin flag	Review matches, review identity documents, edit villages and village pages

How changes are checked and shipped

Automated tests

Suite	Size	What it covers
Unit tests	19 files, 116 cases	GEDCOM parsing and export, phonetic rules and their parity with SQL, privacy helpers, password policy, error messages, tree layout rules
Database tests	17 pgTAP files; 183 assertions passed in the last recorded run, 12 July 2026	Row-level security, living-person masking, search masking, feature building, blocking, scoring, review views and link functions
End-to-end tests	6 Playwright specs	Public pages, page access for guests and users, private items, admin merge, reject and defer, owner accepting a connection

GitHub workflows

Workflow	When it runs	What it does
CI	Every push and pull request	Installs with a frozen lockfile, type-checks, lints and builds the web app
Deploy	After CI passes on main, or by hand	Connects to the server by SSH, resets to main, installs, builds, restarts the service and runs a health check
Integration tests	Pull requests from people with write access	Starts Supabase locally, builds the app and runs Playwright
Release and versions	Changesets and release tags	Version bumps and changelog

> **Pushing to main deploys to production.** Database migrations are not part of the deploy. A migration must be applied to Supabase before the code that needs it reaches main.

Repository layout

apps/web	Next.js app: pages, components, data layer, tests
apps/database/supabase/migrations	48 ordered SQL migrations
apps/database/supabase/tests	pgTAP database tests
apps/database/supabase/demo	Local demo seed: accounts, trees, matches and verifications
docs	Matching engine plan, auth notes, loop log and this report
scripts/walkthrough	Scripts that record the product walkthrough video
.github/workflows	CI, deploy, integration tests, release

What to fix or decide before launch

#	Finding	Why it matters	Suggested action	Priority
1	GEDCOM export includes living people	Any signed-in user can export a public tree, including birth years and villages of living relatives	Filter living people from shared exports; check export rights on the server	High
2	Two definitions of “living”	The people list uses a flag; search uses an age rule. They can disagree	Use the database age rule everywhere	High
3	Tree chart loads life data without masking	Non-members of a public tree may see living people’s details on the chart	Apply the same mask as the people list	High
4	Views of private data are not logged	The privacy policy promises logged access to identity documents	Add an access log and record ID document views first	Medium
5	Access logic lives in several places	Rules can drift between the database helper, the search function and the web code	Make one database function the single source	Medium
6	Contact by request is not built	Families cannot reach each other after a match	Wire the existing relay table to a request screen	Medium
7	No moderator role or appeal path	Every review falls on platform administrators, who can all open ID documents	Decide roles, then add a moderator role with narrower rights	Medium
8	Contribution to the Cause is open for living people	Political details about living people could be published in public trees	Keep the field for deceased people only, or remove it	Medium
9	Matching batch has no schedule	Matches only appear when someone runs the batch	Add a nightly scheduled job	Medium
10	Hosting region and AI provider terms not documented	The law that applies and what providers keep are unknown	Record the region and provider terms in the privacy policy	Medium
11	Privacy contact email is a placeholder	Users cannot exercise their data rights	Set a real address before launch	Medium
12	Project docs are out of date	The README and progress tracker describe an earlier stage	Refresh them from this report	Low

Decisions needed from the steering group

1. **Shared people.** Who approves when one person sits in several family trees.
2. **Villages and places.** Who builds and maintains the list, and which sources count.
3. **Moderators.** How many, which languages, who appoints them, and where an appeal goes.
4. **Hosting.** Which country, which law, and what AI providers may keep.
5. **Contribution to the Cause.** Remove the field, or keep it only for deceased people.

APPENDIX A

Database table inventory

Table	Purpose
profiles	One row per account: display names, language, admin flag
trees	A family tree, its owner and whether it is public
tree_members	Members, roles and access requests with proof
persons	A person in a tree
person_names	Arabic and English names with normalised and phonetic forms
events	Birth, death, marriage and other events with date and place
families	A couple, and the unit children belong to
family_children	Links a child to a family, with pedigree type
places	Villages, towns and cities with districts and coordinates
place_profiles	Village history pages, edited by administrators
person_profiles	Achievements and contribution to the cause
person_attachments	Photos and documents attached to a person
identity_verifications	ID check requests and their review
match_paths	Cached relationship paths, stored without names
name_variants	Dictionary of name spellings
match_features, match_features_dirty	Per-person matching features and the change queue
match_block_keys, match_block_skips	Blocking keys and skipped oversized groups
matches	Scored candidate pairs, admin only
matching_runs, app_settings	Batch run history and engine settings, including the auto-merge switch
person_privacy_holds	Owner opt-outs that exclude a person from matching
eval_pairs, eval_runs	Labelled pairs and results for measuring match quality
person_links, merge_log	Confirmed links between records and the undo log
match_hints	Masked accept or decline queue for tree owners
match_audit	Log of every match status change
notifications	In-app notices for owners
contact_relay	Anonymous, expiring contact tokens; not yet used by the app
overlay_refresh_queue	Deferred refresh of the Palestine Tree overlay
private_items, items, content_blog_posts, content_blog_post_comments	Left over from the starter template; candidates for removal

Views: match_review_cards, v_match_explanations, and the materialised view person_identity_groups that forms the Palestine Tree overlay.

APPENDIX B

Glossary

PFT	Palestine Family Tree. The combined tree formed by linking all individual trees.
IFT	Individual Family Tree. One user’s own tree.
VCC	Village, city or camp. A place of origin.
GEDCOM	The standard file format for exchanging genealogy data. Juthoor uses version 7.
Row-level security	Database rules that decide, row by row, what each user may read or change.
RPC	A database function called through the API.
Fellegi-Sunter	A statistical method for deciding whether two records describe the same person.
Blocking	Grouping records by shared keys so only likely pairs are compared.
pgTAP	A testing framework that runs tests inside PostgreSQL.
Server action	A server function called from a form or button in Next.js.